Djagno 기반의 웹프로그래밍

이 소 영

yisy0703@naver.com

Web Frameworks for Python(https://wiki.python.org/moin/WebFrameworks)
장고 공식 사이트(https://www.djangoproject.com/)
장고 공식 소스 저장소(http://github.com/django/django)
장고 참조 문서(https://docs.djangoproject.com/ko/5.2/)

Agenda

Django(장고) 기반의 파이썬 웹 프로그래밍

Ch01. Django 시작하기
1. Django 란?
2. 개발 환경 구축
3. Django 구조

Ch02. Django App
1. Django Project
2. Model
3. View

Ch03. Model
1. Model 속성 및 옵션
2. Relationship
3. Migrations
4. Admin App

Ch04. Django SQL
1. Django shell
2. Manager & QuerySet
3. 조회 SQL
4. 생성/수정/삭제 SQL
5. Django-Debug-Toolbar

Ch05. Template
1. Template Loader
2. URL Dispatcher
3. Template 상속
4. Template Engines
5. Template Filter

Ch07. Django View
1. View 기본
2. View 활용

Ch06. Django Form
1. HTML form
2. CSRF
3. HttpRequest/HttpResponse
4. Django Form
5. Django Model Form
6. Form Validation

Ch08. File 관리
1. Static Files
2. Media Files

Ch09. 사용자 인증
1. Auth App
2. 회원가입 구현
3. 로그인/아웃 구현
4. Oauth 라이브러리 활용

Ch 05 Template

1. Template Loader
2. URL Dispatcher
3. Template 상속
4. Template Engines
5. Template Filter

Chapter 05. Template

- Template API
  (https://docs.djangoproject.com/en/5.2/ref/templates/api/)

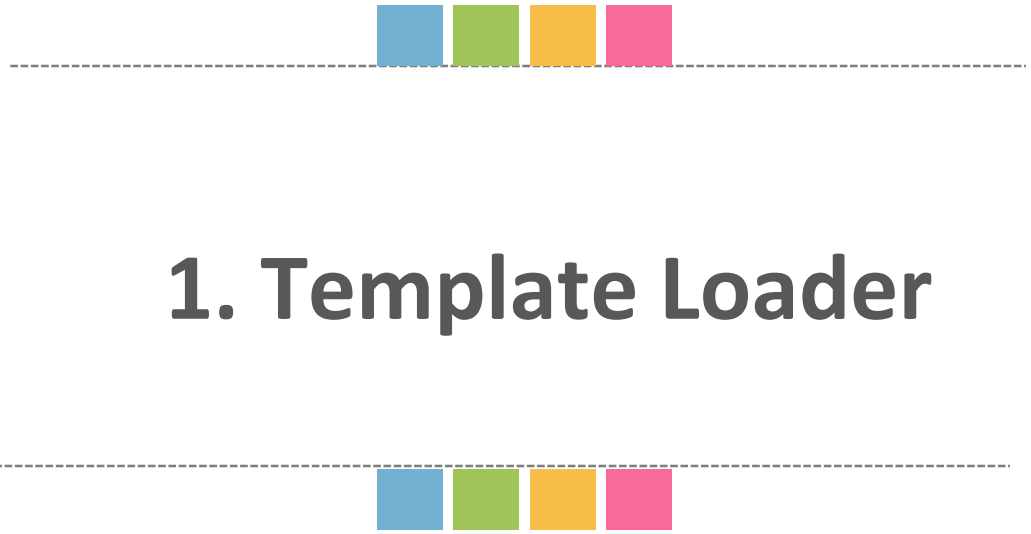
- resolve_url( )
  (https://github.com/django/django/blob/master/django/shortcuts.py#LC119)

- Built-in template tags and filters
  (https://docs.djangoproject.com/en/5.2/ref/templates/builtins)

- Custom template tags and filters
  (https://docs.djangoproject.com/en/5.2/howto/custom-template-tags/)

- Date Format
  (https://docs.djangoproject.com/en/5.2/ref/templates/builtins/#date)

- Time Format
  (https://docs.djangoproject.com/en/5.2/ref/templates/builtins/#time)



1. Template Loader

TEMPLATES

1. Template Loader

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [
                os.path.join(BASE_DIR, 'myproject', 'templates'),
        ],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Django Template Loader

1. django.template.loaders.app_directories.Loader 실행
2. django.template.loaders.filesystem.Loader 실행

- https://docs.djangoproject.com/en/5.2/ref/templates/api
- Template 파일 사용 함수
  - render : HttpResponse 객체 리턴
  - render_to_string : 문자열 리턴

```
response = render(request, 'blog/post_list.html', context_params)

welcome_message = render_to_string('accounts/signup_welcome.txt', context_params)
```

app_directories.Loader

1. Template Loader

- settings.INSTALLED_APPS 에 설정된 앱/templates 에서 템플릿 파일 검색

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'django_extensions',
    'blog',
]
```

/templates

filesystem.Loader

1. Template Loader

- 프로젝트 레벨의 템플릿은 settings.py 에 위치 지정

```
TEMPLATES = [{
    'DIRS': [os.path.join(BASE_DIR,'프로젝트명','templates'), ],
}]
```

- template 파일 검색
  - 앱의 templates을 순서대로 검색
  - 앱에서 검색 실패시 프로젝트의 templates 검색
  - 프로젝트의 templates 검색 실패시 TemplateDoesNotExist 발생



2. URL Dispatcher



Django의 요청 처리 순서

2. URL Dispatcher

1. settings.py의 ROOT_URLCONF 설정 값으로 root URLConf 모듈을 결정함
2. root URLConf 모듈을 로드하고 urlpatterns 변수를 찾음. urlpatterns 설정값은 django.urls.path() 또는 django.urls.re_path() 인스턴스 목록임
3. urlpatterns 순서대로 검색 후 요청 URL과 일치하는 첫번째 패턴에서 멈춤
4. 일치하는 첫번째 패턴의 뷰 함수를 호출함. 호출시 다음 인자를 전달함
   - HttpRequest 인스턴스
   - 이름이 지정되지 않은 인자는 위치 기반으로 전달
   - 키워드 인자는 path() 또는 re_path() 의 kwargs 값에 설정

url pattern

2. URL Dispatcher

- URL에서 view의 인자로 사용되는 값은 꺽쇄 괄호< >를 사용한다.
- URL에서 view의 인자로 사용되는 값의 타입을 지정할 수 있음.
  <int:year>로 지정하면 정수로 처리됨.
- < >를 사용하지 않는 경우는 / 를 제외한 모든 문자열이 일치해야 함.
- 첫번째 /는 추가할 필요 없음. URL 자체적으로 지정되기 때문임.
  "/articles" 대신 "articles"로 지정해야함.



3. Template 상속



Template 상속

- **코드의 재사용성, 유지 보수성 좋음**

- **부모 템플릿**
  - 전체 레이아웃과 자식 템플릿이 재정의 할 block 지정

- **자식 템플릿**
  - 상속받는 템플릿의 block 영역만 재정의 가능
  - block 영역이 아닌 경우는 무시

- **템플릿 상속 문법**
  - {% extends "부모템플릿경로" %}
  - 반드시 첫줄에 위치해야 함

Template 상속을 적용하지 않은 예제


```
# article_list.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" >
    <title>Django Example</title>
</head>
<body>
    <h1>Article</h1>
    <form action="" method="get">
        <input type="text" name="q" value="{{q}}">
        <input type="submit" value="검색">
    </form>
    {% for article in article_list %}
        <li>            <a href="{%url 'article:detail' article.id %}">
            {{article.id}}. {{article.title}}
            </a>
        </li>
    {% endfor %}
    <hr>
    Django Lecture 2022
</body>
</html>
```

Template 상속을 적용하지 않은 예제


```
# article_detail.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Django Example</title>
</head>
<body>
  <h1>Article</h1>
  <h2>{{article.id}}. {{article.title}}</h2>
  {{article.body | linebreaks }}
  <p>
    <a href="#수정">[수정]</a>
    <a href="#삭제">[삭제]</a>
    <a href="{% url 'article:list' %}">[목록]</a>
  </p>
  <hr>
  Django Lecture 2022
</body>
</html>
```

Template 상속을 적용한 예제


```
# myproject/templates/base.html

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Django Example{% endblock %}</title>
</head>
<body>
    <h1>Article</h1>
    {% block content %}
    {% endblock %}
    <hr>
    Django Lecture 2022
</body>
</html>
```

Template 상속을 적용한 예제(자식 template)


```
# article_list.html

{% extends "base.html" %}
{% block title %}
    Artile list
{% endblock %}
{% block content %}
    <form action="" method="get">
        <input type="text" name="q" value="{{q}}">
        <input type="submit" value="검색">
    </form>
    {% for article in article_list %}
        <li>
<!--        <a href="/article/{{article.id}}/detail/">-->
            <a href="{%url 'article:detail' article.id %}">
            {{article.id}}. {{article.title}}
            </a>
        </li>
    {% endfor %}
{% endblock %}
```

Template 상속을 적용한 예제(자식 template)


```
# article_detail.html

{% extends "base.html" %}
{% block title %}
    {{article.title}} 상세보기
{% endblock %}

{% block content %}
  <h2>{{article.id}}. {{article.title}}</h2>
  {{article.body | linebreaks }}
  <p>
    <a href="#수정">[수정]</a>
    <a href="#삭제">[삭제]</a>
    <a href="{% url 'article:list' %}">[목록]</a>
  </p>
{% endblock %}
```

2단계 상속

- 1단계 : 프로젝트 레벨의 레이아웃 템플릿
- 2단계: 각 앱별 레이아웃 템플릿

1. myproject/templates/base.html 작성
2. settings.py

```
TEMPLATES = [ {
    "DIRS": [os.path.join(BASE_DIR, 'myproject', 'templates') ]
 }]
```

3. blog_list.html

```
{% extends “base.html” %}
```



4. Template Engines



Template Engines

- Django Template Engine : Django 기본
- Jinja2 : 써드파트 엔진, django에서 최소한 지원
  - Django Template Engine과 비슷
- Django-jinja
- Mako, HamlPy

Django Template Engines 문법

4. Template Engines

```
{% 태그 인자 %}


{% extends "base.html" %}
{# 파이썬 로직 불가 #}
{% for row in rows %}
    <tr>
        {% for name in row %}
            <td>{{ name }}</td>
        {% endfor %}
    </tr>
{% endfor %}
```

Variables

4. Template Engines

- {{ first_name }}
- {{ mydict.key }} : dict의 key에 attr처럼 접근
- {{ myobj.attr }}
- {{ myobj.func }} : 함수호출. 인자있는 함수호출 불가
- {{ mylist.0 }} : 인덱스 접근도 attr처럼 접근

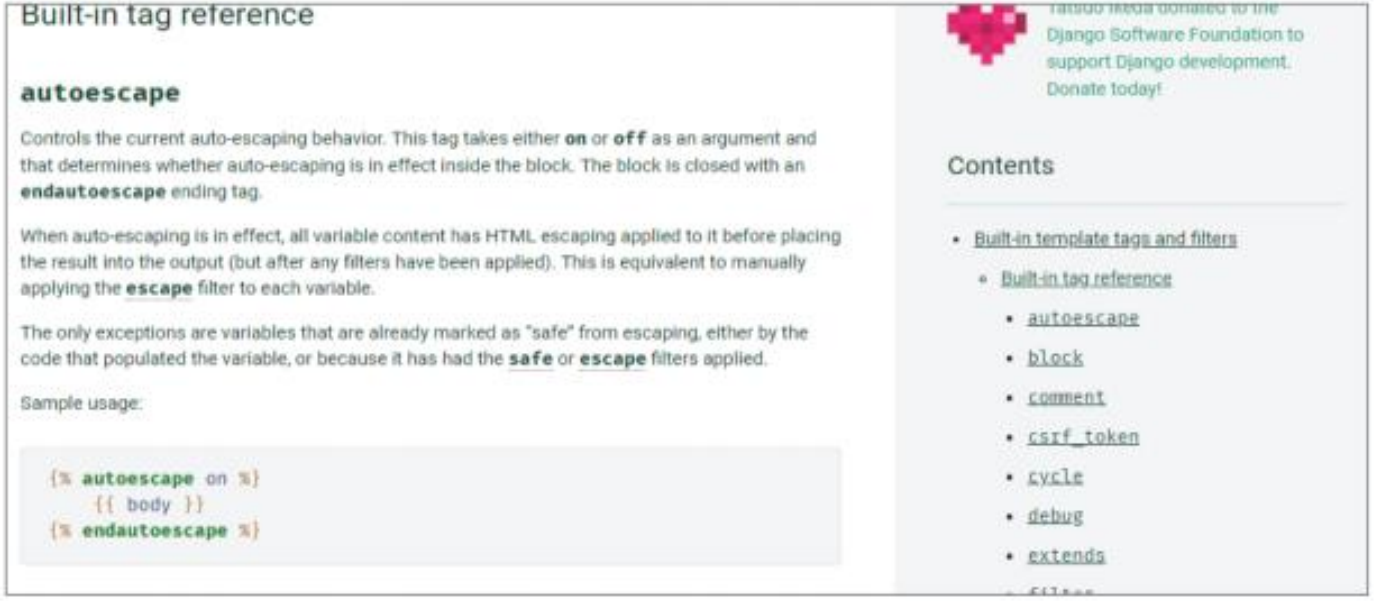
```
people = {'Amy':23, 'Josh':25}
people['Amy']   / {{ people.Amy }}

class Person(object):
        def say_hello(self):
                print('hello')
person.say_hello() / {{ person.say_hello }}

names=['Amy','Josh']
names[0]  / {{ names.0 }}
```

Django Template Tag

- Django Templates 용 함수
- 문법 : {% %}
- 빌트인 태그 지원 및 커스텀 태그 추가 가능
- 참조 문서
  - https://docs.djangoproject.com/ko/5.2/ref/templates/builtins



Built-in tag reference

autoescape

Controls the current auto-escaping behavior. This tag takes either **on** or **off** as an argument and that determines whether auto-escaping is in effect inside the block. The block is closed with an **endautoescape** ending tag.

When auto-escaping is in effect, all variable content has HTML escaping applied to it before placing the result into the output (but after any filters have been applied). This is equivalent to manually applying the **escape** filter to each variable.

The only exceptions are variables that are already marked as "safe" from escaping, either by the code that populated the variable, or because it has had the **safe** or **escape** filters applied.

Sample usage:

```
{% autoescape on %}
    {{ body }}
{% endautoescape %}
```

Tatsuo Ikeda donated to the Django Software Foundation to support Django development. Donate today!

Contents

- Built-in template tags and filters
  - Built-in tag reference
    - autoescape
    - block
    - comment
    - csrf_token
    - cycle
    - debug
    - extends

block 태그

- 템플릿 상속에서 사용
- 자식 템플릿이 오버라이딩할 block 영역 정의
- 자식 템플릿은 부모가 정의한 block 만 재정의 가능

```
{% block block-name %}
        block 내에 내용을 쓰실 수 있습니다.
{% endblock %}
```

comment 태그

- 템플릿 주석 지정
- comment 영역은 서버 단에서 처리하지 않음
- 여러줄 주석 처리

```
{% comment "Optional note" %}
  주석 1
  주석 2
{% endcomment %}

{# 한줄 주석 #}
```

csrf_token 태그

- Cross Site Request Forgeries를 막기 위해 CSRF Middleware 제공

```
45  MIDDLEWARE = [
46      'django.middleware.security.SecurityMiddleware',
47      'django.contrib.sessions.middleware.SessionMiddleware',
48      'django.middleware.common.CommonMiddleware',
49      'django.middleware.csrf.CsrfViewMiddleware',
50      'django.contrib.auth.middleware.AuthenticationMiddleware',
51      'django.contrib.messages.middleware.MessageMiddleware',
52      'django.middleware.clickjacking.XFrameOptionsMiddleware',
53  ]
```

- POST 요청시 CSRF 토큰 체크를 함
- csrf_token 태그로 CSRF 토큰 발급

csrf_token 태그


```
<form method="POST" action="">
    {% csrf_token %}
    <input type="text" name="author" />
    <textarea name="message"></textarea>
    <input type="submit" />
</form>
```

extends 태그

- 자식 템플릿에서 부모 템플릿 상속
- 소스의 첫번째줄에 위치해야 함
- 자식 템플릿에서는 상속받는 부모 템플릿의 block만 정의할 수 있음

```
{% extends "base.html" %}
```

for 태그

- 지정 객체를 순회
- 파이썬의 for문과 동일

```
<ul>
    {% for article in article_list %}
            <li>{{ article.title }}</li>
    {% endfor %}
</ul>
```

반복문안에서 변수 사용

- forloop.counter : 1부터 시작하여 1씩 증가 index
- forloop.counter0 : 0부터 시작하여 1씩 증가 index
- forloop.revcounter : 끝인덱스부터 시작, 1씩 감소 index
- forloop.revcounter0 : 끝에서부터 1씩 감소 index
- forloop.first , forloop.last : 첫 실행 , 마지막 실행 여부
- forloop.parentloop : 중첩 loop에서 부모 loop를 지정

```
#views.py
def test(request):
    people = ['Amy', 'Josh', 'Tobey', 'John']
    return render(request, 'mytest/test.html', {'people':people})

#test.html
{% for p in people%}
{{forloop.counter}} : {{ p }} <br/>
{% endfor %}
```

for ... empty 태그

- for문내에서 지정 object를 찾을 수 없거나, 비었을 때 empty block이 수행

```
<ul>
{% for athlete in athlete_list %}
    <li>{{ athlete.name }}</li>
{% empty %}
    <li>Sorry, no athletes in this list.</li>
{% endfor %}
</ul>
```

```
#views.py
def test(request):
    person_list = []
    return render(request, 'mytest/test.html', {'person_list':person_list})

#test.html
{% for p in person_list %}
  {{ p.name }}
{% empty %}
  no name
{% endfor %}
```

for ... empty 태그


```
athelete_list = []

## 파이썬 코드
if athelete_list:
   for athelete in athelete_list:
         print(athelete.name)
else
   print("empty")

## 템플릿 코드
{% for athelete in athelete_list %}
     {{ athelete.name }}
{% empty %}
     empty
{% endfor %>

## 템플릿 동일 코드
{% if athelete_list %}
    {% for athelete in athelete_list %}
          {{ athelete.name }}
{% else %}
        empty
{% endif %>
```

if 태그

- 파이썬의 if문과 동일

```
{% if athlete_list %}
    Number of athletes: {{ athlete_list|length }}
{% elif athlete_in_locker_room_list %}
    Athletes should be out of the locker room soon!
{% else %}
    No athletes.
{% endif %}
```

include 태그

- 다른 템플릿을 로딩/렌더링을 수행하며, 현재 context가 그대로 전달

- include 시에 keyword 인자를 지정하여, 추가 context 지정 가능

```
{% include "bar.html" %}
{% include "names.html" with person="Jane" greeting="Hello" %}
```

lorem 태그

- 랜덤 채우기 텍스트를 생성

```
{% lorem [count] [method] [random] %}
```

- count : 생성할 단락/단어의 수 (디폴트 : 1)
- method : 단어 - w, HTML 단락- p, Plain Text 단락- b 지정 (디폴트 : b)
- random : random 지정/미지정

```
{% lorem %} : 보통의 채우기 텍스트 출력
{% lorem 3 p %} : HTML 단락 3개 출력
{% lorem 2 w random %} : 랜덤 단어 2개 출력
```

now 태그

- 현재 날짜/시간 출력
  - It is {% now “jS F Y H:I” %}
- 출력 포맷 참조
  - https://docs.djangoproject.com/en/5.2/ref/templates/builtins/#std:templatefilter-date

url 태그

- URL Reverse를 수행한 URL문자열을 출력
- 인자처리는 django.shortcuts.resolve_url 함수와 유사하게 처리하나, get_absolute_url 처리는 하지 않음

```
{% url "some-url-name-1" %}
{% url "some-url-name-2" arg arg2 %}
{% url "some-url-name-2" arg arg2 as the_url %}
```



5. Template Filter



Template Filter

- **템플릿 변수값 변환을 위한 함수, | (파이프 기호) 사용**
  - {{ var|filter1 }} : var 은 filter1의 첫번째 인자로 사용
  - {{ var|filter2:var2 }} : var은 filter2의 첫번째 인자, var2은 두번째 인자
- **다중 필터 함수 연결 가능**
  - {{ var|filter3:var2|filter4 }} : var은 filter3의 첫번째 인자, var2는 두번째 인자이며 filter3에서 처리된 결과는 filter4의 첫번째 인자
- **빌트인 Filter 지원, 장고앱별로 커스텀 Filter 추가 가능**

date/time filter

- 지정 포맷으로 출력
- date/time 참조 문서
  - https://docs.djangoproject.com/en/5.2/ref/templates/builtins/#date
  - https://docs.djangoproject.com/en/5.2/ref/templates/builtins/#time
- Django 기본값
  - django.conf.global_settings.py

```
332   # Default formatting for date objects. See all available format strings here:
333   # https://docs.djangoproject.com/en/dev/ref/templates/builtins/#date
334   DATE_FORMAT = 'N j, Y'
335
336   # Default formatting for datetime objects. See all available format strings here:
337   # https://docs.djangoproject.com/en/dev/ref/templates/builtins/#date
338   DATETIME_FORMAT = 'N j, Y, P'
339
340   # Default formatting for time objects. See all available format strings here:
341   # https://docs.djangoproject.com/en/dev/ref/templates/builtins/#date
342   TIME_FORMAT = 'P'
```

date/time filter

5. Template Filter

```
{{ datetime_obj|date:"D d M Y" }}
=> 'Wed 09 Jan 2008'
{{ datetime_obj|date:"DATE_FORMAT" }}
=> 디폴트 'N j, Y' (e.g. Feb. 4, 2003)
{{ datetime_obj|date:"DATETIME_FORMAT" }}
=> 디폴트 'N j, Y, P' (e.g. Feb. 4, 2003, 4 p.m.)
{{ datetime_obj|date:"SHORT_DATE_FORMAT" }}
=> 디폴트 'm/d/Y' (e.g. 12/31/2003)
{{ datetime_obj|date:"SHORT_DATETIME_FORMAT" }}
=> 디폴트 'm/d/Y P' (e.g. 12/31/2003 4 p.m.)
{{ datetime_obj|time:"TIME_FORMAT" }}
=> 디폴트 'P' (e.g. 4 p.m.)
```

```
from django.utils import timezone

timezone.now()
datetime.datetime(2018, 6, 26, 8, 20, 26, 567729, tzinfo=<UTC>)
timezone.now().strftime("%Y-%m-%d %H:%M:%S")
'2018-06-26 08:21:04'
```

timesince/timeuntil filter


```
{{ past_dt|timesince }} => 현재시각 기준 (now - past_dt)
{{ past_dt|timesince:criteria_dt }} => 기준시각 기준 (criteria_dt - past_dt)
{{ future_dt|timeuntil }} => 현재시각 기준 (future_dt - now)
{{ future_dt|timeuntil:past_dt}} => 기준시각 기준 (future_dt - past_dt)
```

- **timesince**
  - 과거부터 현재까지 시간

- **timeuntil**
  - 미래 시점까지 남아있는 시간

date/time filter

5. Template Filter

```
# views.py
def test(request):
    past_dt = timezone.datetime(1971,8,22,0,0)
    criteria_dt = timezone.datetime(2001,3,19,0,0)
    future_dt = timezone.datetime(2037,1,1,0,0)

      return render(request, 'mytest/test.html', 'datetime_obj':now,
                                                 'past_dt':past_dt,
                                                 'criteria_dt':criteria_dt,
                                                 'future_dt':future_dt})
```

```
#test.html
{{ datetime_obj|date:"D d M Y" }} <br/>
{{ datetime_obj|date:"DATE_FORMAT" }} <br/>
{{ datetime_obj|date:"DATETIME_FORMAT" }} <br/>
{{ datetime_obj|date:"SHORT_DATE_FORMAT" }} <br/>
{{ datetime_obj|date:"SHORT_DATETIME_FORMAT" }} <br/>
{{ datetime_obj|time:"TIME_FORMAT" }} <br/>

{{ past_dt|timesince }} <br/>
{{ past_dt|timesince:criteria_dt }} <br/>
{{ future_dt|timeuntil }} <br/>
{{ future_dt|timeuntil:past_dt}} <br/>
```

timezone

- timezone 지정 객체

```
from django.utils import timezone
timezone.now()                                          #timezone 정보
datetime.datetime(2018, 6, 26, 8, 20, 26, 567729, tzinfo=<UTC>)

import datetime
datetime.datetime.now()                                 #timezone 정보없음
datetime.datetime(2018, 6, 26, 17, 39, 50, 233286)

datetime.datetime(2018,6,26,0,0) # UTC
datetime.datetime(2018,6,26,9,0) # Asia/Seoul
datetime.datetime(2018,6,26,0,0) == datetime.datetime(2018,6,26,9,0)
False
```

- django.utils의 timezone 사용 권장

Template Filter

- **join**
  - 순회가능한 객체를 지정 문자열로 연결
  - 파이썬의 **str.join(list)과 동일**
    - {{ people|join:" // " }} => ['a', 'b', 'c'] 일 경우, "a // b // c"를 출력

- **length**
  - value의 길이를 출력
  - 파이썬의 **len(value)과 동일**
  - Undefined 변수일 경우 **0을 출력**
    - {{ people|length }} => ['a', 'b', 'c', 'd'] 일 경우 4 를 출력

Template Filter

- **truncatechars**
  - 문자열을 지정 글자 갯수까지만 보여주고 .... 추가

- **truncatechars_html**
  - HTML 태그를 보호하면서 문자열을 지정 글자 갯수까지만 보여주고 .... 추가

```
value1 = "Joel is a slug"
value2 = "<p>Joel is a slug</p>"

{{ value1|truncatechars:9 }} => "Joel i..."
{{ value2|truncatechars_html:9 }} => "<p>Joel i...</p>"
```

Template Filter

- **truncatewords**
  - 문자열을 지정 단어 갯수까지 보여주고 … 추가
- **truncatewords_html**
  - HTML 태그를 보호하면서 문자열을 지정 단어 갯수까지 보여주고 … 추가

```
value1 = "Joel is a slug"
value2 = "<p>Joel is a slug</p>"

{{ value1|truncatewords:2 }} => "Joel is ..."
{{ value2|truncatewords_html:2 }} => "<p>Joel is ...</p>"
```

- **urlencode**
  - 지정값을 urlencode 처리

```
value = "https://www.example.org/foo?a=b&c=d"
{{ value|urlencode }} => "https%3A//www.example.org/foo%3Fa%3Db%26c%3Dd"
```